Project Report: Brain Tumor Detection

Name: Vaibhav Bharti

Registration No.: 25BCE10081

Course: Fundamentals of AIML

Course Code: CSA0001(B11+B12+B13)

Faculty: Prof. J. Manikandan

1. Problem Identification

Brain tumors represent one of the most severe forms of oncological conditions, where the growth of abnormal cells within the skull can lead to life-altering neurological deficits or mortality. The primary diagnostic tool for these conditions is Magnetic Resonance Imaging (MRI). However, the manual analysis of these scans presents several critical bottlenecks:

- **Subjectivity:** Interpretation can vary between different radiologists based on experience.
- **Volume:** The sheer number of scans generated daily can lead to diagnostic delays.
- **Fatigue:** Prolonged manual screening increases the risk of overlooking subtle pathological markers.

2. Clinical Significance

Integrating Machine Learning (ML) into the clinical workflow serves as a 'force multiplier' for medical professionals. An automated detection system provides an objective, repeatable, and high-speed preliminary screening. This matters because early-stage detection often dictates the viability of treatment options like minimally invasive surgery or targeted radiotherapy. In underserved regions, such systems can bridge the gap between high patient volume and low specialist availability.

3. Technical Approach & Architecture

3.1 Data Pipeline and Augmentation

Medical imaging data is notoriously scarce. To ensure the model generalizes across different MRI machines and patient orientations, we implemented a robust **ImageDataGenerator** pipeline. This included real-time transformations:

Transformation	Rational for Medical Context
Rescaling (1/255)	Normalizes pixel intensity to ensure weights don't explode during training.
Rotation & Shear	Simulates slight variations in patient head positioning during the scan.
Horizontal Flip	Accounts for the bilateral symmetry of the brain while doubling dataset size.

3.2 Model Topology

The architecture follows a Sequential CNN pattern, utilizing three hierarchical blocks to decompose the image from raw pixels into abstract medical features. Each block contains a **Conv2D** layer for feature mapping and a **MaxPooling2D** layer for spatial down-sampling. The final classifier uses a 128-unit Dense layer followed by a Sigmoid output to determine the probability of tumor presence.

4. Strategic Design Decisions

Optimization Algorithm: We selected the 'Adam' optimizer due to its adaptive learning rate properties, which is particularly effective for non-convex optimization problems like image classification.

Regularization Strategy: Overfitting is a high risk in medical AI. We implemented a 50% Dropout layer. By randomly zeroing out half of the neurons during training, we force the network to develop redundant, robust pathways rather than relying on specific 'noise' in the training images.

5. Challenges and Mitigation

Challenge 1: High False Negatives. In healthcare, missing a tumor (False Negative) is more dangerous than a false alarm. *Mitigation:* While the current model uses Binary Crossentropy, future iterations could implement weighted loss functions to penalize False Negatives more heavily.

Challenge 2: Image Noise. MRI scans often contain artifacts. *Mitigation:* The use of Max-Pooling helped the model become invariant to small-scale noise, focusing only on the most salient features.

6. Lessons Learned & Future Scope

The project demonstrated that deep learning can effectively identify pathological patterns in MRI data. A key takeaway was the critical importance of the validation split—relying purely on training accuracy can be deceptive. Future work will involve **Transfer Learning** using pre-trained weights from VGG16 or ResNet to leverage features learned from millions of general images, potentially increasing accuracy even further.

7. About Project

Domain: Medical Imaging & Deep Learning

Model Architecture: Convolutional Neural Networks (CNN)

Language: Python (TensorFlow/Keras)

8. References

Kaggle dataset: Brain MRI Images for Tumor Detection

Scikit-learn Documentation

Python Documentation

9. Code Screenshot

```python
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import matplotlib.pyplot as plt
import os

# 1. Configuration and Hyperparameters
IMG_SIZE = (224, 224)
BATCH_SIZE = 32
EPOCHS = 20
# Replace with the path to your dataset
DATASET_PATH = 'path_to_your_dataset'

# 2. Data Preprocessing & Augmentation
# Augmentation helps the model generalize better by creating variations of the images
datagen = ImageDataGenerator(
    rescale=1./255,
    validation_split=0.2, # 20% for validation
    rotation_range=10,
    width_shift_range=0.1,
    height_shift_range=0.1,
    shear_range=0.1,
    zoom_range=0.1,
    horizontal_flip=True,
    fill_mode='nearest'
)

train_generator = datagen.flow_from_directory(
    DATASET_PATH,
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='binary',
    subset='training'
)

validation_generator = datagen.flow_from_directory(
```

```python
    DATASET_PATH,
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='binary',
    subset='validation'
)

# 3. Building the CNN Model
def build_model():
    model = models.Sequential([
        # First Convolutional Block
        layers.Conv2D(32, (3, 3), activation='relu', input_shape=(224, 224, 3)),
        layers.MaxPooling2D(2, 2),

        # Second Convolutional Block
        layers.Conv2D(64, (3, 3), activation='relu'),
        layers.MaxPooling2D(2, 2),

        # Third Convolutional Block
        layers.Conv2D(128, (3, 3), activation='relu'),
        layers.MaxPooling2D(2, 2),

        # Flattening and Dense Layers
        layers.Flatten(),
        layers.Dense(128, activation='relu'),
        layers.Dropout(0.5), # Reduces overfitting
        layers.Dense(1, activation='sigmoid') # Binary output: Tumor or No Tumor
    ])

    model.compile(optimizer='adam',
                  loss='binary_crossentropy',
                  metrics=['accuracy'])
    return model

model = build_model()
model.summary()
```

```python
# 4. Training the Model
history = model.fit(
    train_generator,
    epochs=EPOCHS,
    validation_data=validation_generator
)

# 5. Visualizing Results
def plot_history(history):
    acc = history.history['accuracy']
    val_acc = history.history['val_accuracy']
    loss = history.history['loss']
    val_loss = history.history['val_loss']
    epochs_range = range(len(acc))

    plt.figure(figsize=(12, 5))
    plt.subplot(1, 2, 1)
    plt.plot(epochs_range, acc, label='Training Accuracy')
    plt.plot(epochs_range, val_acc, label='Validation Accuracy')
    plt.legend(loc='lower right')
    plt.title('Training and Validation Accuracy')

    plt.subplot(1, 2, 2)
    plt.plot(epochs_range, loss, label='Training Loss')
    plt.plot(epochs_range, val_loss, label='Validation Loss')
    plt.legend(loc='upper right')
    plt.title('Training and Validation Loss')
    plt.show()

plot_history(history)

# 6. Save the Model
model.save('brain_tumor_detector.h5')
```